

LLM-Lasso : A Robust Framework for Domain-Informed Feature Selection and Regularization

Erica Zhang, Ryunosuke Goto, Naomi Sagan, Jurik Mutter, Nick Phillips, Ash Alizadeh, Kangwook Lee, Jose Blanchet, Mert Pilanci, Robert Tibshirani

Introduction

- **Feature selection**
 - Cornerstone of statistical learning
 - Reducing complexity and improving interpretability
 - Widespread : Lasso approach

Introduction

- **Lasso approach**

- 일반적인 선형 회귀에 ℓ_1 regularization를 추가한 형태
- 불필요한 특징의 계수를 0으로 만듦

- Objective function

:

$$\min_{\beta} \left\{ \underbrace{\frac{1}{2} \sum_{i=1}^n (y_i - \beta_0 - x_i^\top \beta)^2}_{\text{SSE}} + \lambda \underbrace{\sum_{j=1}^p |\beta_j|}_{\ell_1 \text{ penalty}} \right\}.$$

SSE

ℓ_1 penalty

Introduction

- **Lasso approach**

- Problem

: traditional Lasso approach는 오직 training data에 기반,
feature selection 과정에서 expert knowledge 고려 X

⇒ **LLM-powered feature selection**

: **LLM-derived penalty factors + Lasso penalty terms**

LLM-Lasso

- 메타 데이터에서 얻은 정보를 반영하는 Regularization 항을 추가해서, 모델이 학습할 때 이 정보를 고려

$$\hat{\theta}_{\tilde{f}} := \tilde{f}(\mathcal{D}, \mathcal{D}_{\text{meta}}) = \arg \min_{\theta} \mathcal{L}(\theta, \mathcal{D}) + \lambda R(\theta, \mathcal{D}_{\text{meta}})$$

- Notation

$\mathcal{D}$: 기존 데이터셋, $\mathcal{D}_{\text{meta}}$: 추가적인 메타 데이터, θ : prediction parameter

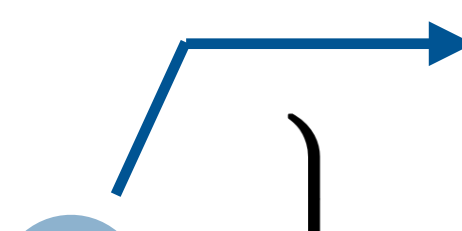
λ : regularization parameter, $R(\bullet)$: regularizer

LLM-Lasso

- **Two key components of LLM-Lasso framework**
 - (i) core statistical model that integrates an LLM-informed penalty into Lasso
 - LLM-informed penalty
 - (ii) general pipeline for training a task-specific LLM on expert knowledge base
 - Prompt engineering
 - Knowledge-base embedding via RAG

LLM-Lasso

- **LLM-informed penalty**

$$\min_{\beta} \left\{ \frac{1}{2} \sum_{i=1}^n (y_i - \beta_0 - x_i^\top \beta)^2 + \lambda \sum_{j=1}^p w_j |\beta_j| \right\}$$


- **Notation**

λ : regularization parameter

w_j : penalty factors to each coefficient

1. Inverse importance penalty factors

$$w_j = (I_j)^{-\eta}$$

- I_j : importance score for feature j
- η 값은 cross-validation을 통해 결정

2. **ReLU-form penalty factors**

$$\tilde{w}(j) = 1 + \frac{(j - (1 - \gamma)p)^+}{\gamma p} \cdot (\tilde{w}(p) - 1)$$

- LLM에서 생성된 penalty factor와 기존 Lasso 회귀에서 사용되는 ℓ_1 -norm 기반 penalty factor를 interpolating
- $\tilde{w}(j)$: LLM이 결정한 중요도 점수에 ReLU 처리를 한 특징 j 의 penalty factor
- γ : ReLU threshold
- $(j - (1 - \gamma)p)$: penalty를 받지 않는 상위 중요 특징의 수
- γ 값은 cross-validation을 통해 결정

LLM-Lasso

- **Prompt engineering**
 - full prompt template for all classification tasks

$$\mathcal{P}^{\text{full}} = \text{prompt}(\mathcal{Q}^{\text{user}}(\mathcal{A}(\phi, c)), \mathcal{C}^{\text{retriever}}(k, \mathcal{R}(\phi, c)), \mathcal{H}^{\text{system}})$$

User query

$A(\bullet)$: task description prompt
 ϕ : feature, c : category

Top k retrieved contexts

$R(\bullet)$: retrieval prompt
w/ retrieval knowledge base

Summarization
of past queries,
responses

- use greedy decoding
 - due to its simplicity and deterministic behavior
- chain-of-thought (CoT) prompting

LLM-Lasso

- **Prompt engineering**
 - components of prompt engineering
 - : A (task description), and R (retrieval prompt)

$$\mathcal{P}^{\text{full}} = \text{prompt}(Q^{\text{user}}(\mathcal{A}(\phi, c)), \mathcal{C}^{\text{retriever}}(k, \mathcal{R}(\phi, c)), \mathcal{H}^{\text{system}})$$

User query

$A(\bullet)$: task description prompt
 ϕ : feature, c : category

Top k retrieved contexts

$R(\bullet)$: retrieval prompt
w/ retrieval knowledge base

**Summarization
of past queries,
responses**

LLM-Lasso

- **Knowledge-base embedding via RAG**
 - RAG
 - Retrieval + Augmented + Generation
 - DB(추가적인 knowledge-base)에서 relevant document 참조

LLM-Lasso

- **Knowledge-base embedding via RAG**
 - RAG (1) Preprocessing

$$\{d_i\}_{i=1}^N = \{E(D_i)\}_{i=1}^N, \quad E : \text{Text} \rightarrow \mathbb{R}^d$$

- $D_{i=1}^N$: N 개의 text documents로 구성된 knowledge base
- use the OpenAI embeddings off-the-shelf
- apply Hierarchical Navigable Small World(HNSW) algorithm, implemented in chromadb

LLM-Lasso

- **Knowledge-base embedding via RAG**
 - RAG (2) Retrieval

$$\text{Sim}(q, d_i) = \frac{q^T d_i}{\|q\|_2 \|d_i\|_2}$$

- $q \in \mathbb{R}^d$: query vector, $\{d_i\}_{i=1}^N$: stored embeddings
- top k documents with the highest semantic similarity scores are retrieved
→ supplied as context C

Experiment

- **Robustness**
 - perform adversarial data corruption simulations
 - observed LLM hallucination
 - remain robust compared with LLM-Score

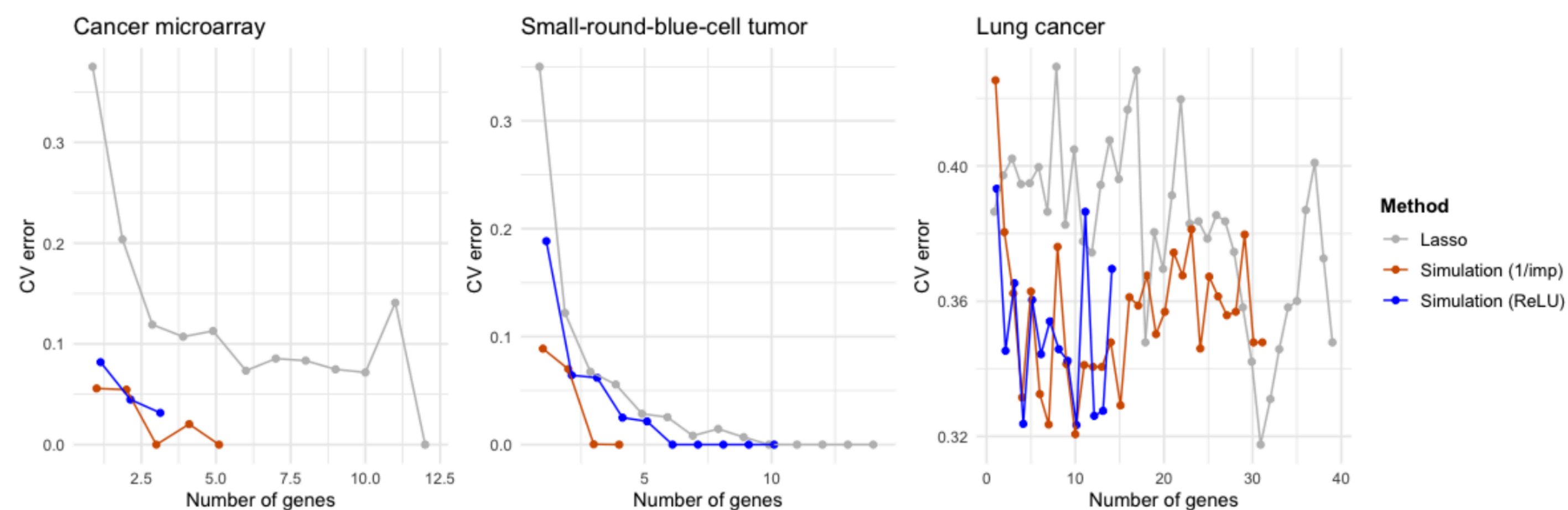
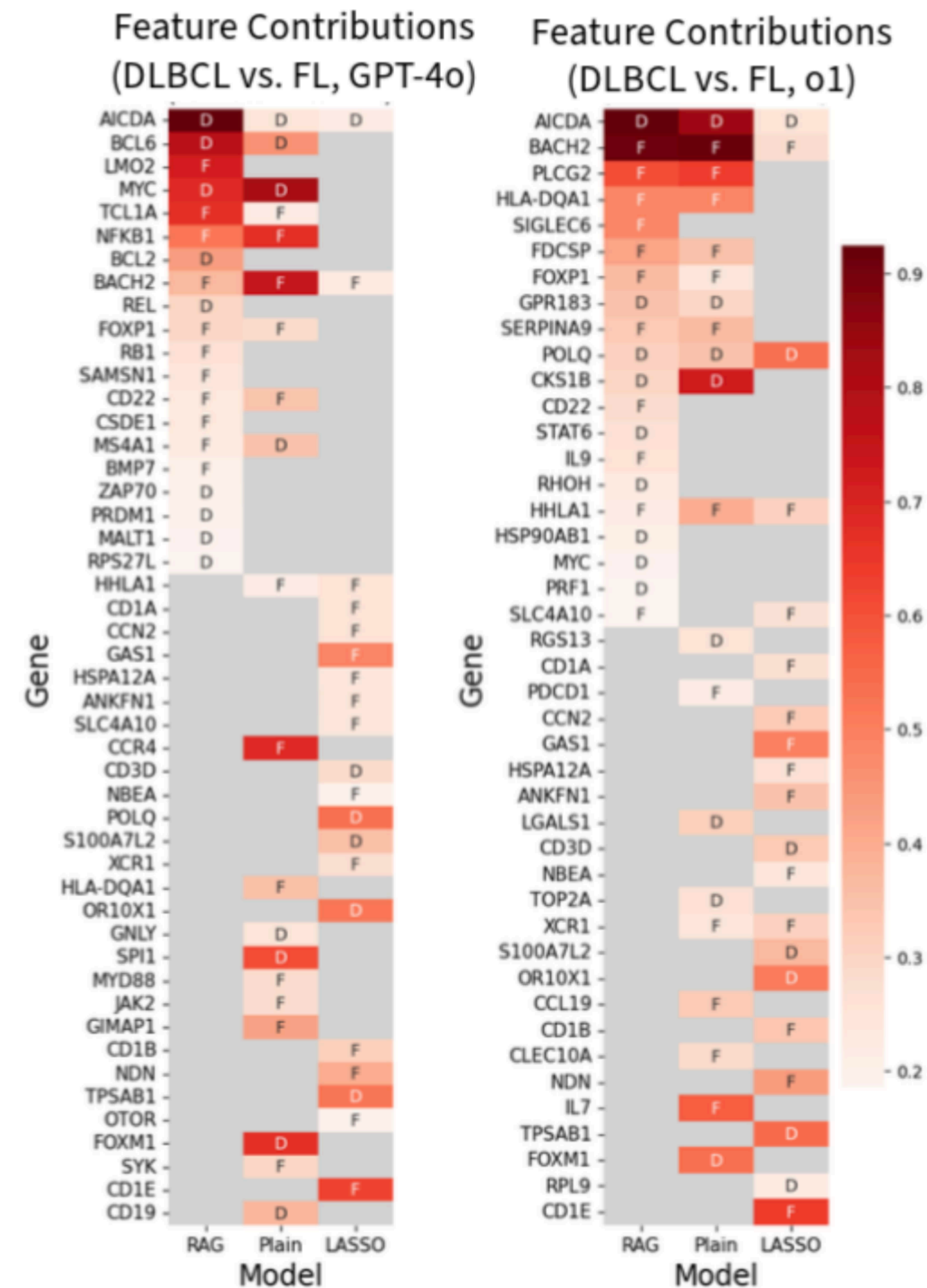


Figure 9. Test error in simulations

Experiment

- **Feature contribution metric**
 - 여러번의 LLM-Lasso regression 실험을 진행했기 때문에, a single list of selected features and their coefficients 추출 어려움
 - Feature contribution metric : proportion that each feature appears across the full path of the number of features
 - letters in the heatmaps
 - :“F” coefficients in the direction of FL and “D” for DLBCL



Conclusion

- Introducing LLM-Lasso
 - LLM-powered feature selection
 - effectively combines contextual knowledge with data-driven insight
- Providing clues to identify important features
- Protects inaccuracies, hallucinations
- Task specific LLM-Lasso's clinical applicability